

Uncertainty-Aware Clarification in LLM Agents with Information Gain

Mengyi Deng¹ Zhiwei Li¹ Xin Li¹ Tingyu Zhu¹ Ying Zhao² Zhijiang Guo^{1,3} Wei Wang^{1,3}

Abstract

Large Language Model (LLM) agents often operate under underspecified user instructions, where latent uncertainty over user intent leads to erroneous tool actions. To address this challenge, we propose a goal-oriented clarification framework that aligns clarification behavior with ambiguity resolution. Central to our approach is the Information Gain Reward, a metric that quantifies the utility of clarification questions by measuring the Bayesian belief update towards the ground-truth goal induced by the clarification exchange. We train the clarifier (LLM) using this reward to optimize for high information gain, ensuring that clarifications effectively reduce uncertainty and improve task completion within the agent-tool-user environment. We validate our framework within a clarification-enhanced τ -Bench environment, conducting cross-agent evaluations across five heterogeneous backbones. Empirical results demonstrate that our method consistently improves the success rate by 3.7% over the no-clarification baseline, while adding only 0.3 total interaction steps on average.

1. Introduction

Large Language Model (LLM) agents have recently shown significant proficiency in utilizing external tools and performing multi-step decision-making across a diverse set of real-world applications (Qu et al., 2025; Yao et al., 2022; Wölflein et al., 2025). Through their interaction with various external tools, these agents are capable of completing complex tasks that involve planning, reasoning, and iterative feedback (Wölflein et al., 2025). In practical interactive contexts, however, user instructions are often *underspecified* or vague, leaving essential aspects of the user’s true intent

¹ Information Hub, HKUST(GZ) ² Jilin University ³ HKUST . Correspondence to: Wei Wang <weiwcs@ust.hk>, Zhijiang Guo <zhijiangguo@hkust-gz.edu.cn>, Mengyi Deng <mdeng974@connect.hkust-gz.edu.cn>.

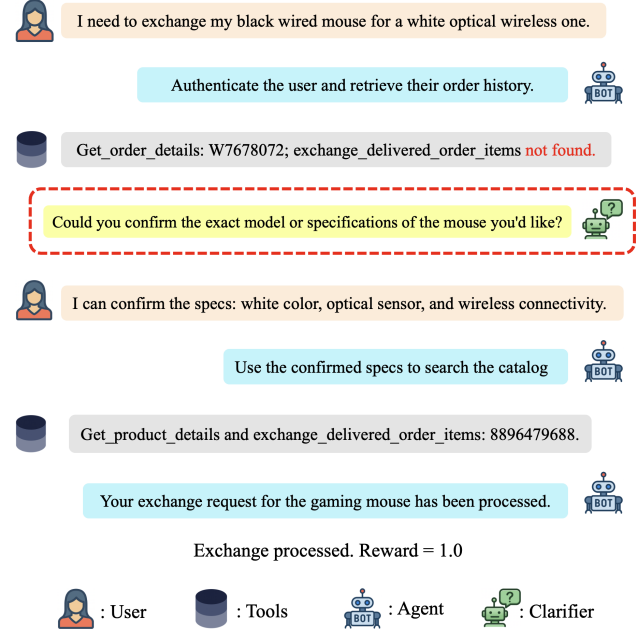


Figure 1. An example of clarification in a τ -retail trajectory. When an initial tool call fails due to missing or underspecified information, the Clarifier poses a targeted follow-up question (highlighted) to elicit the required constraints from the user. The additional information provided through this exchange allows the agent to proceed with a corrected tool invocation and complete the task.

implicit (Yehudai et al., 2025; Qi et al., 2025). Such ambiguous user demands present a fundamental challenge for tool-using agents: premature or inaccurate tool actions may lead to irreversible mistakes, fragile execution paths, and task failure (Wang et al., 2024).

A common approach to addressing underspecified user instructions is issuing clarification questions, as shown in Figure 1, to resolve ambiguity before action (Clark, 1996; Zou et al., 2023). While human assistants naturally identify missing information and seek clarification, this ability remains underdeveloped in LLM-based agents (Rao & Daumé III, 2018). A key challenge is that LLMs are primarily trained to answer questions rather than to determine when clarification is necessary or how to formulate it. Consequently, existing training pipelines provide scant guidance on the necessity or effectiveness of clarification (Fu et al., 2020; Ouyang et al., 2022), often leaving such behaviors implicit or entangled

with internal reasoning (Suri et al., 2025). Moreover, the difficulty in training LLMs for clarification stems from a misalignment in supervision: existing annotations typically reward surface-level fluency while failing to capture a question’s capacity to reduce requirement uncertainty (Zhang et al., 2023). Without an explicit signal for uncertainty reduction, models default to favoring generic, safe questions that provide limited informational value (Szymanski et al., 2025; Son et al., 2024), struggling to evolve toward truly targeted information-seeking behaviors.

To address these challenges and move beyond subjective supervision, we reframe the clarification problem within the context of uncertainty-aware learning. Motivated by this perspective, we propose an information-theoretic framework that quantifies the utility of a clarification question by measuring its impact on reducing uncertainty about the user’s true goal. Specifically, we model the clarification process as a Bayesian belief update and quantify the utility of a question by measuring the shift in the model’s probability mass towards the ground-truth goal following the clarification exchange. This formulation provides a performance-oriented objective that directly links questioning strategies to effective ambiguity resolution. We then utilize this intrinsic reward signal to drive Decoupled Advantage Policy Optimization (DAPO) (Yu et al., 2025), guiding the agent to evolve questioning strategies that are explicitly optimized for missing information recovery rather than surface-level conversational behaviors.

This paper studies clarification as an execution-grounded information acquisition problem, where the agent intervenes only when asking a question is expected to reduce uncertainty about the latent user goal and improve downstream tool use. We evaluate this formulation in a Clarifier-augmented τ -Bench setting, where user intent is partially observable and tool feedback exposes execution failures, requiring the agent to recover through interactive clarification rather than direct access to the full goal. We further validate the framework through comprehensive analyses of training dynamics, the upper bounds of agent and clarification effectiveness, trigger frequency, and cross-agent generalization.

2. Related Work

Tool-Augmented Agents. Large language models have increasingly been framed as tool-using agents capable of performing multi-step reasoning and decision-making. These agents utilize various processes, such as planning (Yao et al., 2022; Schick et al., 2023), tool invocation (Yuan et al., 2025; Zhu et al., 2025; Wu et al., 2024b), and feedback-driven execution (Lu et al., 2025; Wu et al., 2025; Liu et al., 2023b). To evaluate agent behavior in complex, tool-rich settings, various benchmarks and environments have been proposed. For

example, (Qu et al., 2025) focuses on tool efficiency, (Liu et al., 2023a) provides a broad framework for agent task evaluation, and (Andriushchenko et al., 2024) addresses robustness in adversarial contexts. These diverse benchmarks (Wang et al., 2024; Andriushchenko et al., 2024; Xu et al., 2024; Li et al., 2024b; Wu et al., 2024a) offer valuable insights into agent performance across different application domains. However, while existing agent frameworks primarily optimize tool invocation for well-defined tasks, they lack explicit mechanisms to handle ambiguity in real-world instructions, often entangling clarification with execution. To address this, our work leverages the dynamic interaction capabilities of τ -Bench (Yao et al., 2024) to explicitly optimize clarification strategies.

Clarification Learning and Alignment Signals. Clarification questions serve as a critical mechanism for resolving ambiguity. While foundational studies emphasized their necessity for grounding in human communication (Clark, 1996; Traum & Larsson, 2003; Yizhou et al., 2024; Zhang et al., 2024), recent advances have extended this mechanism to conversational search (Aliannejadi et al., 2019) and active uncertainty reduction strategies (Zou et al., 2023; Xia et al., 2025; Li et al., 2024a). While current efforts primarily focus on evaluation (Zhou et al., 2025) or static question selection (Choudhury et al., 2025), they often overlook the learning of clarification policies for tool-using agents. In parallel, IGPO (Wang et al., 2025a) formulates information-gain-based optimization for multi-turn agents under the assumption that the target answer is known in advance. Compared with existing methods operate with predefined answer targets or fixed clarification candidates, our setting considers tool-using agents that infer latent user goals online, condition on tool state and execution feedback, handle free-form user responses, and learn an amortized clarification policy rather than only selecting from static questions or optimizing against a known answer slot.

Although emerging policy optimization frameworks like GRPO (Shao et al., 2024; Wang et al., 2025b) and DAPO (Yu et al., 2025) offer a robust mechanism to learn such behaviors, applying them to clarification is challenging: standard alignment approaches typically rely on human or LLM-based judges that prioritize surface-level text quality over informational value (Chaudhari et al., 2025). To address this, we propose an information-theoretic framework that explicitly measures a question’s ability to resolve underspecified user intent. Specifically, we optimize for Expected Information Gain (EIG) to quantify goal uncertainty reduction. This enables policy optimization to explicitly target uncertainty reduction, maximizing information gain to facilitate successful task execution.

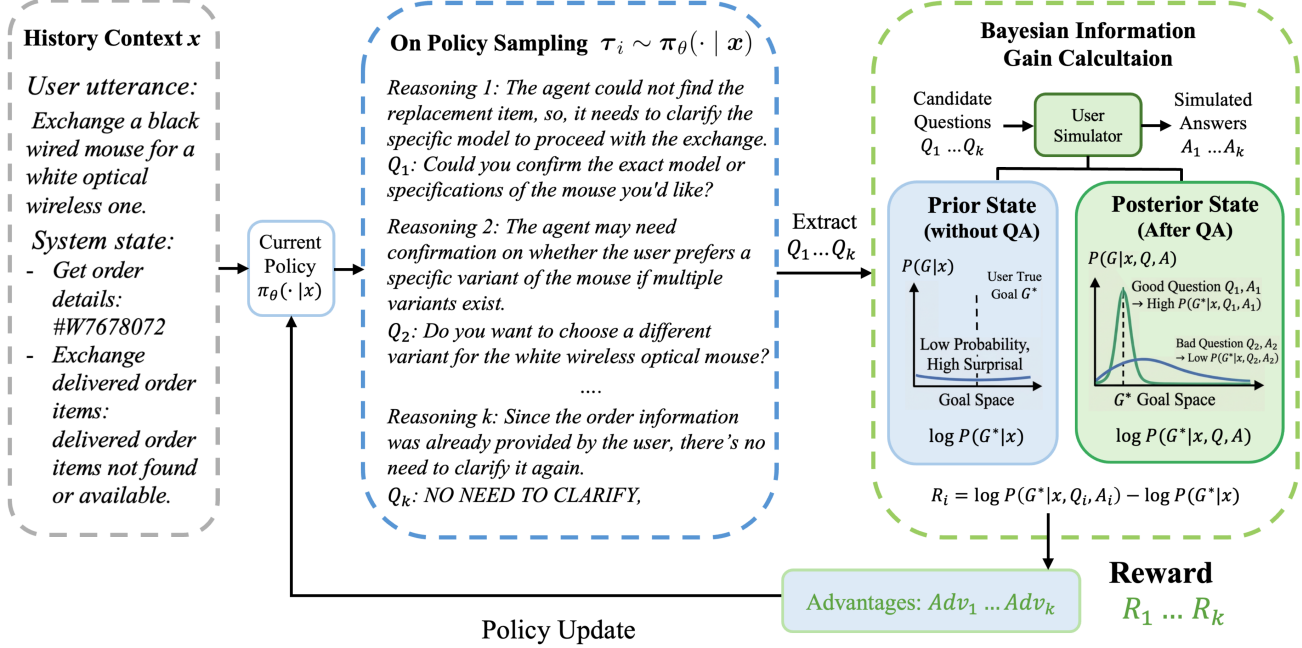


Figure 2. Overview of the Amortized Bayesian Experimental Design framework. The model performs on-policy sampling to generate candidate questions. These candidates are evaluated by our Belief Update Reward, which quantifies efficacy as the shift in the teacher-forced log-likelihood of the ground-truth goal G^* (Bayesian Belief Update). Acting as an amortized surrogate for the intractable expected information gain, this signal guides the DAPO update, steering the policy towards clarifications that maximally reduce goal uncertainty.

3. Method

In this section, we present a framework for learning alignment-driven clarification policies. We first introduce the execution-grounded interaction protocol within τ -Bench (Section 3.1), followed by the data construction process (Section 3.2). Section 3.3 details the Amortized Bayesian Experimental Design and its role in formulating the Information Gain Reward, while Section 3.4 explains how the DAPO algorithm uses this reward to optimize the clarifier for resolving task ambiguity.

3.1. Environment and Interaction Protocol

We conduct our study on τ -Bench (Yao et al., 2024), which provides a partially observable Markov decision process (POMDP) based environment for tool-agent interaction. To handle user ambiguity, we extend this interaction loop with a pluggable and LLM-based clarifier module, as illustrated in Figure 1. When the agent issues a tool call $a_t \in \mathcal{A}_{\text{tool}}$ and receives the corresponding observation o_t , the clarifier assesses whether the execution feedback reveals latent ambiguity or missing task-critical information. If so, it generates a concise follow-up question; otherwise, no clarification turn is produced. We focus on learning a clarification policy that optimizes the decision of whether to intervene and what information to solicit based on execution feedback. This

process yields an interaction trajectory defined as:

$$\tau = \left(U, (a_t, o_t, Q_t, A_t)_{t=1}^T, G \right),$$

where U denotes the initial fuzzy instruction and G represents the user’s latent ground-truth goal. While G remains unobserved by the agent during interaction, it guides the user simulator’s responses and serves as the reference for computing our information-gain rewards. At each step t , the agent executes a tool action a_t and observes o_t . Conditioned on this execution feedback, the clarifier may issue a clarification question Q_t and receive a corresponding response A_t from the user simulator before the next action is taken. If no clarification is made at step t , we set $Q_t = A_t = \emptyset$. These trajectories serve as the foundation for optimizing our clarifier policy.

3.2. Data Construction

To curate the dataset for policy optimization, we process interaction trajectories from τ -Bench, restricting our scope to the first 16 agent steps to focus on ambiguity resolution. From this filtered subset, we retain a final dataset of 2,676 step-level instances, where each data point consists of the dialogue history x and a task-defined user goal G provided by τ -Bench. We further normalize the raw user goals to obtain a clean, canonical goal representation G^* , which serves as the supervision target for reward computation throughout

this paper. Details of the data construction process and the goal transfer from G to G^* are provided in Appendix 6.1.

3.3. Amortized Bayesian Experimental Design

To learn *when* and *what* to clarify, we formulate clarification generation from a Bayesian Experimental Design (BED) (Chaloner & Verdinelli, 1995) perspective and train a clarifier to propose questions that are expected to reduce uncertainty about the user’s latent goal G^* . Rather than directly optimizing this objective, which is intractable in interactive settings, we approximate the experimental design principle through an amortized training pipeline.

Theoretical Objective. The amortized training pipeline described above can be viewed as an approximation to the following idealized Bayesian objective. Formally, let G^* be the ground-truth goal, x be the interaction context, and $P(A | Q, x)$ denote the answer distribution induced by the user simulator after question Q is asked in context x . An optimal clarification Q^* maximizes the *Expected Information Gain (EIG)*:

$$I(G^*; A | Q, x) = \mathbb{E}_{A \sim P(\cdot | Q, x)} [H(G^* | x) - H(G^* | x, Q, A)] \quad (1)$$

where $H(\cdot)$ denotes entropy. Direct maximization of this objective is computationally intractable. Specifically, calculating the EIG requires integrating over the vast, open-ended space of latent user goals and potential responses to estimate the posterior entropy. Such high-dimensional inference is prohibitively expensive for every clarification candidate.

Amortized Optimization via Belief Update. To address this, we propose an **amortized** approach. We optimize a question policy $\pi_\theta(Q | x)$ to internalize the experimental design process and directly generate high-information questions. Separately, we use the same model in teacher-forcing mode as a belief scorer $P_\theta(G^* | \cdot)$, with gradients detached during reward computation. We define the reward R_t as a *pointwise* information-gain proxy. To ensure the reward is invariant to goal complexity, we formulate it as the shift in the length-normalized teacher-forced log-likelihood of G^* :

$$R_t(x_t, Q_t, A_t) = \underbrace{\frac{1}{L} \sum_{j=1}^L \log P_\theta(g_j^* | x_t, Q_t, A_t, g_{<j}^*)}_{\text{Posterior Belief}} - \underbrace{\frac{1}{L} \sum_{j=1}^L \log P_\theta(g_j^* | x_t, g_{<j}^*)}_{\text{Prior Belief}} \quad (2)$$

Here, $G^* = (g_1^*, \dots, g_L^*)$ represents the token sequence of the ground-truth goal. The term $\log P_\theta(\cdot)$ denotes the probability of the next token computed via teacher forcing. By normalizing these log-probabilities (dividing by length

L), R_t quantifies the Bayesian Belief Update: a positive value indicates that the clarification exchange (Q_t, A_t) has effectively concentrated the model’s probability mass on the true goal sequence. To estimate this belief update in practice, any sufficiently capable LLM could be used to estimate these likelihood terms. In this work, we use the same backbone as the clarifier policy to maintain consistency between belief representation and policy behavior. Since the policy is trained to reduce its own uncertainty over the user goal, evaluating likelihood shifts within the same model ensures that the reward reflects the policy’s internal belief update.

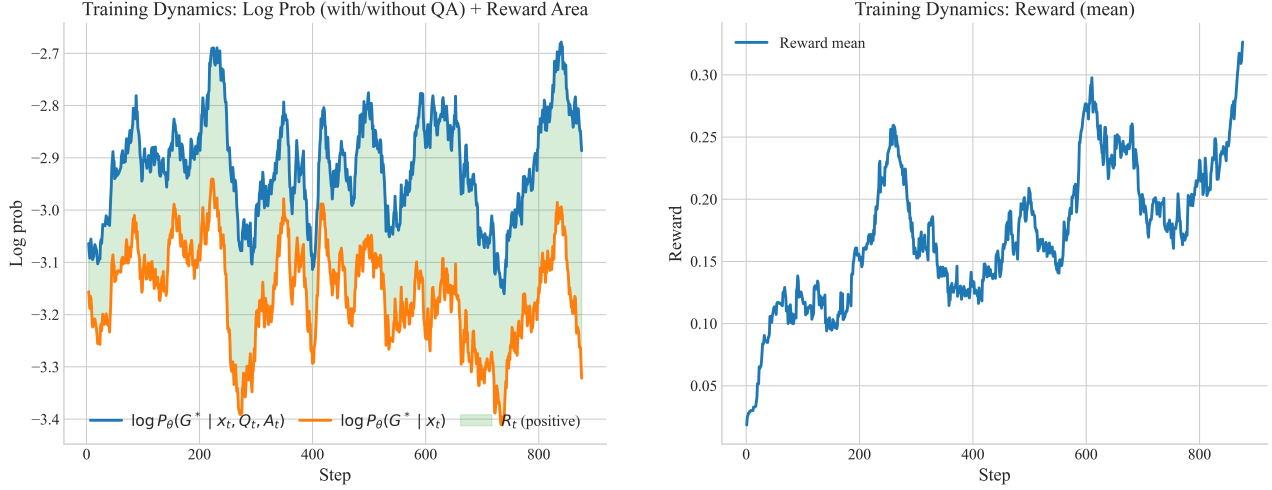
Mathematically, as derived in Appendix 6.2, this reward corresponds to the Pointwise Mutual Information (PMI) between the response and the ground truth. It can be expressed as the log-importance weight $\log \frac{P(A|G^*, Q, x)}{P(A|Q, x)}$, which effectively quantifies how much the observed answer A precisely distinguishes the true goal G^* from the general hypothesis space. Crucially, when the clarifier decides not to intervene (i.e., No need to ask token), the posterior belief remains identical to the prior, naturally yielding a reward of zero ($R_t = 0$).

Strict User Simulator for Training. The validity of the belief update relies heavily on the quality of the simulated response A_t . If the user simulator is too helpful (revealing the goal even for generic questions), the clarifier tends to hack the reward without learning valid clarification strategies. Therefore, we employ a Strict User Simulator during training. Unlike the relaxed user prompt used in τ -Bench, our strict simulator reveals task-specific information *only* when the question Q_t is specific and relevant. This ensures that high rewards are assigned exclusively to questions that genuinely necessitate clarification. A detailed comparison of the simulator prompts is provided in Appendix 6.3.4.

3.4. Optimization via DAPO

As illustrated in Figure 2, the clarifier operates on the current interaction context x , which consists of the accumulated dialogue and execution history and serves as the input to the clarifier LLM. Conditioned on this context, the policy performs on-policy sampling to generate a set of candidate clarification questions $\{Q_1, \dots, Q_K\}$. Each candidate is answered by the Strict User Simulator, yielding responses $\{A_1, \dots, A_K\}$. We evaluate the resulting trajectories using the Belief Update Reward R_t derived in Eq. 2. Finally, these rewards are converted into group-relative advantages to update the policy via DAPO (Yu et al., 2025), maximizing the expected cumulative information gain.

On-Policy Monte Carlo Estimation. Our training process can be interpreted as a Monte Carlo estimation of the expected Bayesian utility. The policy $\pi_\theta(\cdot | x_t)$ functions as a proposal distribution, sampling a group of K candidate



(a) Log probabilities with and without clarification; the shaded difference denotes the information gain reward.

(b) Training dynamics of the mean reward under the proposed information gain.

Figure 3. Training dynamics of DAPO under the information gain reward objective.

questions $\{Q^{(k)}\}_{k=1}^K$, and $A^{(k)}$ is the response returned by the strict user simulator for the sampled question $Q^{(k)}$. The empirical average of their rewards provides an unbiased estimate of the expected utility:

$$\mathcal{U}(\theta) \approx \mathbb{E}_{Q \sim \pi_\theta}[R_t] \approx \frac{1}{K} \sum_{k=1}^K R_t(x_t, Q^{(k)}, A^{(k)}). \quad (3)$$

As training progresses, π_θ increasingly concentrates probability mass on the regions of the question space that yield high information gain.

Advantage Computation. To reduce variance and stabilize training, we compute the group-relative advantage for each candidate k :

$$Adv_k = \frac{R_t^{(k)} - \mu_K(R_t)}{\sigma_K(R_t) + \epsilon}, \quad (4)$$

where μ_K and σ_K denote the mean and standard deviation across the sampled group, and ϵ is a small constant for numerical stability. Since R_t is a sequence-level metric derived from the final belief update, we broadcast the scalar advantage Adv_k to all tokens within the question $Q^{(k)}$.

DAPO Objective. We optimize the policy using the decoupled clipping objective, which is particularly effective for the sparse reward nature of clarification tasks:

$$\mathcal{J}(\theta) = \frac{1}{\sum_k |Q^{(k)}|} \sum_{k=1}^K \sum_{j=1}^{|Q^{(k)}|} \min \left(\rho_{k,j} Adv_k, \text{clip}(\rho_{k,j}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) Adv_k \right) \quad (5)$$

where $\rho_{k,j}$ is the importance sampling ratio. The clipping function restricts $\rho_{k,j}$ within $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$ (with $\epsilon_{\text{high}} > \epsilon_{\text{low}}$), effectively raising the ceiling for exploration on high-potential tokens. By maximizing this objective, the clarifier learns to align its clarifying behavior with the goal of maximizing information gain, effectively amortizing the cost of optimal experimental design.

4. Experiments

Building upon the framework introduced in Section 3, we evaluate our approach within the clarifier-augmented τ -Bench environment. To ensure a clean separation between training and evaluation, all models are fine-tuned *exclusively* on a dataset of 500 τ -retail training trajectories. Testing is subsequently conducted on two distinct sets: 115 held-out retail tasks, which represent the in-domain setting, and 50 airline tasks, which represent the out-of-distribution (OOD) setting. The `airline` domain serves as a particularly challenging testbed for transfer learning, as it features task structures, tool APIs, and user interaction patterns that differ markedly from those observed during training. Further implementation details and specific training hyperparameters are provided in Appendix 6.6.

4.1. Training Dynamics and Performance

We first analyze the training dynamics to assess whether optimization steers the agent toward uncertainty-reducing behaviors. Figure 3(a) decomposes our Information Gain Reward into two teacher-forced log-likelihood terms: the prior confidence in the ground-truth goal given the interaction context alone, $\log P(G^* | x)$, and the posterior confidence after in-

Table 1. Ablation study on τ -Bench (Retail and Airline) based on training with Qwen3-1.7B. Pass@1 (%) with deterministic decoding (temperature = 0), averaged over three runs. Numbers in parentheses indicate changes relative to *None*. *Baseline* is the untrained Qwen3-1.7B clarifier; *LLM as a Judge* replaces the information-gain reward with a question-quality judge; *w/o Information Gain* optimizes the posterior likelihood term without the prior-posterior contrast.

Method	Success Rate (% $\uparrow$)			Steps $\downarrow$		
Strategy	Retail	Airline	Average	Average (w/o Clarifier)	Average Clarify	Sum
None	16.5	14.3	15.4	24.2	0.0	24.2
Baseline (Qwen3-1.7B)	16.5 (+0.0)	13.3 (-1.0)	14.9 (-0.5)	23.9	4.2	28.1
LLM as a Judge	18.3 (+1.8)	8.0 (-6.3)	13.2 (-2.2)	26.7	2.9	29.6
w/o Information Gain	15.6 (-0.9)	10.0 (-4.3)	12.8 (-2.6)	26.8	2.4	29.2
Ours	18.3 (+1.8)	17.3 (+3.0)	17.8 (+2.4)	23.8	1.3	25.1

Notes. $\uparrow$ ($\downarrow$) indicates higher (lower) is better. *Average* denotes the mean across domains. *Average (w/o Clarifier)* counts agent and user steps excluding clarifier turns; *Average Clarify* denotes the average number of clarifier invocations; *Sum* counts all *Average (w/o Clarifier)* and *Average Clarify* steps. w/o Information Gain removes the uncertainty reduction term and optimizes $\log P(G \mid x_t, Q_t, A_t)$ only.

Table 2. Exploration Boundaries of Agents and Clarifier: Impact of Different Clarification Strategies on Task Completion Performance. Numbers in parentheses indicate changes relative to the agent’s performance under the *None* condition (fuzzy user intent without clarification). $\uparrow$ ($\downarrow$) indicates higher (lower) is better. *None (Full User Intent)* represents the condition where full user intent is provided without clarification, reflecting the agent’s performance boundaries.

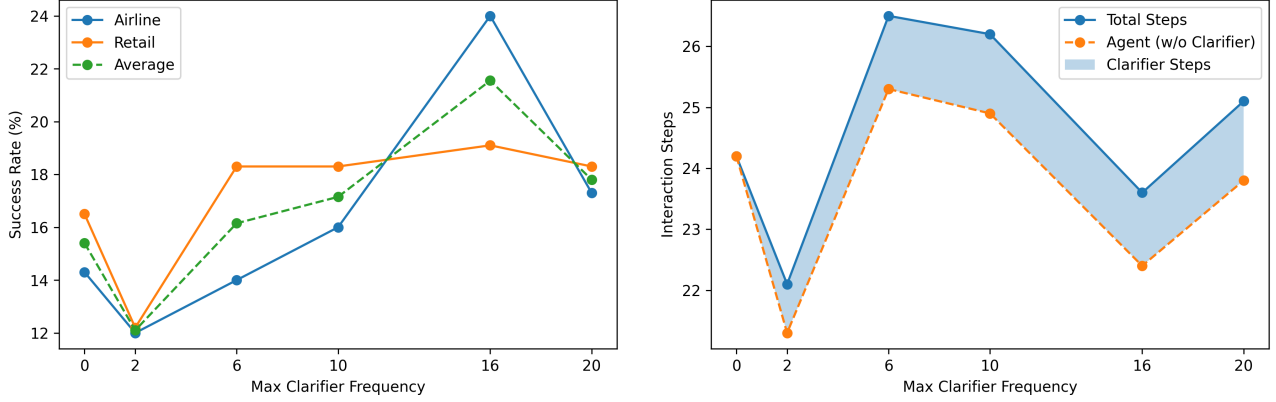
Method	Success Rate (% $\uparrow$)			Steps $\downarrow$		
Clarify Strategy	Retail	Airline	Average	Average (w/o Clarifier)	Average Clarify	Sum
None	16.5	14.3	15.4	24.2	0.0	24.2
Qwen3-8B	15.7 (-0.8)	8.0 (-6.3)	11.9 (-3.5)	23.1	2.7	25.8
Qwen3-14B	16.5 (+0.0)	16.0 (+1.7)	16.3 (+0.9)	24.2	5.1	29.3
Qwen3-30B	15.7 (-0.8)	19.0 (+4.7)	17.4 (+2.0)	21	3.8	24.8
DeepSeek-R1	14.8 (-1.7)	19.0 (+4.7)	16.9 (+1.5)	20.5	3.9	24.4
DeepSeek-V3.1	19.1 (+2.6)	17.0 (+2.7)	18.1 (+2.7)	21	3.9	24.9
Ours (Qwen3-1.7B-trained)	18.3 (+1.8)	17.3 (+3.0)	17.8 (+2.4)	23.8	1.3	25.1
None (Full User Intent)	20	16	18	22.3	0.0	22.3

incorporating the clarification exchange, $\log P(G^* \mid x, Q, A)$. Notably, the likelihood conditioned on the interaction context without clarification, $\log P(G^* \mid x)$, remains mildly oscillatory throughout training, reflecting variation in sampled contexts during on-policy training. However, $\log P(G^* \mid x)$ remains substantially below the $\log P(G^* \mid x, Q, A)$ posterior curve, and the gap between the two curves tends to widen with training. This pattern suggests that the learned policy improves primarily by eliciting additional task-relevant evidence, not by simply increasing prior confidence in ambiguous contexts.

Consistent with this interpretation, Figure 3(b) shows that the mean Information Gain Reward increases steadily over training, rising from a near-zero baseline to approximately 0.15 by step 200 and continuing to improve thereafter. This monotonic trend, without signs of collapse or oscillatory behavior, suggests that the reward provides a stable optimization signal. Together with the divergence observed in Figure 3(a), these results indicate that DAPO effectively exploits the information gain objective to refine the clarification policy, progressively improving the agent’s ability to acquire informative feedback.

Table 1 reports an ablation study comparing different clarification training strategies on τ -Bench. Compared to the pre-trained Qwen3-1.7B clarifier, our method consistently improves success rates on both domains, increasing performance from 16.5% to 18.3% on *retail* and from 13.3% to 17.3% on *airline*, while simultaneously reducing interaction cost. In particular, the average number of clarifier invocations drops sharply from 4.2 to 1.3 per task, resulting in a 3 reduction in total interaction steps, indicating that the learned policy asks fewer but more targeted clarification questions. When the available information is already sufficient, the prior belief over user intent is already highly concentrated. In this case, additional clarification often introduces weakly informative or irrelevant context, making the prediction of the ground-truth action sequence under teacher forcing more difficult and leading to a decrease in log-likelihood. Intuitively, unnecessary questions introduce distracting information without providing meaningful constraints on the underlying intent, and are therefore less favorable than abstention (reward = 0).

In contrast, removing the information gain term and optimizing with a likelihood-only objective (w/o Information Gain)



(a) Success rate on the `airline` and `retail` domains, and their average, under different maximum clarification budgets.

(b) Average number of interaction steps, including clarification steps (shaded area), across the `airline` and `retail` domains.

Figure 4. Effect of clarification budget on task success and interaction efficiency under forced sampling.

leads to a substantial performance degradation, especially on the `airline` domain, where success rate falls from 17.3% to 10%, suggesting that conditional likelihood alone is insufficient to encourage informative clarifications in domains. The LLM-as-a-judge variant further exhibits unstable behavior: although it matches our method on `retail` (18.3%), performance collapses on `airline` (8%) and incurs higher interaction costs, highlighting the limitation of stylistic or preference-based supervision in capturing the task-specific informational value of clarification questions. Details of the LLM-as-a-judge prompt template are provided in Appendix 6.3.5.

4.2. Exploration of Clarifier Boundaries

Table 2 compares our specialized 1.7B clarifier against a wide range of large-scale LLMs (from 8B to 671B). Despite the significant disparity in parameter scale, our method achieves a task success rate of 17.8%, approaching the best-performing large backbone (DeepSeek-V3.1, 18.1%) with a negligible margin of merely **0.3%**. This result demonstrates that a small, information-driven model can effectively match the utility of significantly larger models in resolving task-oriented ambiguity.

Crucially, this performance is achieved with superior interaction efficiency. While general-purpose models tend to over-clarify with 3.9 to 5.1 turns per task, our policy converges with just **1.3** turns on average. This efficient strategy is vital for real-world deployment, as it mitigates the deleterious effects of over-clarification, including interactional redundancy, user fatigue, and the risk of task state pollution (where excessive dialogue history distracts the agent). By optimizing for Information Gain, our clarifier strikes a favorable balance, facilitating ambiguity resolution while reducing the cognitive and temporal costs of interaction.

Comparing against the *Full User Intent* offers a profound insight into information modality (see Appendix 6.3.6 for the specific prompt template). While this static oracle serves as an upper bound of agent’s capability, our interactive approach surprisingly outperforms it in the complex `Airline` domain (17.3% vs. 16%). This suggests that in intricate scenarios, interactive ambiguity resolution is superior to static information loading. By breaking down complex constraints into sequential turns, our clarifier mitigates the risk of information overload, enabling the agent to attend to task-critical details more effectively than when overwhelmed by a monolithic instruction dump. Overall, our learned clarifier effectively approximates this theoretical performance boundary (17.8% vs. 18% on average) while incurring a marginal overhead of only 2.8 total steps.

4.3. Impact of Clarification Trigger Frequency

Figure 4(a) examines how the maximum clarification budget affects Pass@1 success rates on the `airline` and `retail` domains. Introducing clarification consistently improves performance over the no-clarification setting, demonstrating its effectiveness in resolving underspecified user intents. Increasing the budget from 0 to a moderate level yields a substantial gain in average success rate, rising from approximately 15% to over 21%. However, the relationship between clarification budget and performance is clearly non-monotonic: success rates peak at intermediate budgets and plateau or decline as the budget increases further, with the effect being most pronounced in the `airline` domain. This finding challenges the common design assumption that more clarification is inherently beneficial. Instead, it suggests that excessive clarification introduces context pollution and redundant interactions, which can disrupt the agent’s reasoning flow without adding task-relevant constraints.

Table 3. Agent and clarifier generalization performance with the clarifier decoded deterministically. Different agent backbones are compared under varying clarification strategies, including no clarification (*None*), a *Qwen3-8B* clarifier, and a learned 1.7B clarifier (*Ours*) trained with our DAPO setting. $\uparrow$ ($\downarrow$) indicates higher (lower) is better. The best results for each model are bolded. The values in parentheses indicate the change relative to the performance with no clarification *None*.

Setting		Success Rate (% $\uparrow$)			Steps $\downarrow$		
Agent Model	Clarifier Model	Retail	Airline	Average	Average (w/o Clarifier)	Clarify	Sum
Qwen3-8B	None	16.5	14.3	15.4	24.2	0	24.2
	Qwen3-8B	15.7 (-0.8)	8.0 (-6.3)	11.9 (-3.5)	23.1	2.7	25.8
	Ours	18.3 (+1.8)	17.3 (+3.0)	17.8 (+2.4)	23.8	1.3	25.1
Qwen3-32B	None	23.4	14	18.7	24	0	24
	Qwen3-8B	21.7 (-1.7)	10 (-4.0)	15.9 (-2.6)	20.4	5.1	25.5
	Ours	24.3 (+0.9)	18 (+4.0)	21.2 (+2.5)	16.9	2.6	19.5
DeepSeek-R1	None	31.3	40	35.7	18.1	0	18.1
	Qwen3-8B	26.0 (-5.3)	42 (+2.0)	34 (-1.7)	11.5	1	12.5
	Ours	33.9 (+2.6)	46 (+6.0)	40 (+4.3)	21.8	0.5	22.3
DeepSeek-V3.2	None	36.5	30	33.3	23.5	0	23.5
	Qwen3-8B	39.1 (+2.6)	40 (+10)	39.6 (+6.3)	23.3	2.7	26
	Ours	37.4 (+0.9)	42 (+12)	39.7 (+6.4)	23.1	0.7	23.8
GLM-4-32B	None	6.1	24	15	33.3	0	33.3
	Qwen3-8B	7 (+0.9)	20 (-4.0)	13.5 (-1.5)	35.6	1.5	37.1
	Ours	7 (+0.9)	26 (+2.0)	16.5 (+1.5)	32.9	1	33.9
Agent Avg.	None	22.8	24.5	23.6	24.6	0	24.6
	Qwen3-8B	21.9 (-0.9)	24 (-0.5)	23 (-0.6)	22.8	2.6	25.6
	Ours	24.2 (+1.4)	29.9 (+5.4)	27.3 (+3.7)	23.7	1.2	24.9

Figure 4(b) reveals a relationship between clarification frequency and task efficiency. Initially, increasing the budget from 0 to 2 reduces total interaction steps from 24.2 to 22.1 without improving the success rate, indicating that insufficient questioning fails to resolve core task ambiguities. A significant efficiency gain occurs as the budget scales from 6 to 16: while the success rate grows consistently, total steps peak at 26.5 (budget 6) before declining to 23.6 (budget 16). This trend demonstrates that effective mid-stage clarification successfully converts into substantial performance gains while streamlining the execution path by preempting redundant tool actions. Beyond budget 16, interaction steps rise to 25.1 with no commensurate success rate gains. This reversal suggests that excessive clarification becomes counterproductive, introducing informational noise that distracts the agent rather than facilitating goal resolution. To further understand the remaining failure modes, we conduct a qualitative audit of representative trajectories and distinguish clarification failures from downstream execution errors; detailed analyses and case studies are provided in Appendix 6.5.

4.4. Cross-Agent Generalization Ability

Table 3 presents the cross-agent generalization performance under deterministic decoding across a diverse set of agents, including Qwen3-32B, DeepSeek-R1, DeepSeek-V3.2, and GLM-4-32B. We evaluate three clarification strategies: no clarification (*None*), a publicly released model (*Qwen3-8B*)

as a clarifier, and our learned clarifier (*Ours*), which is trained using information-gain-driven DAPO.

Overall trends. Averaged across all five agent backbones, our method consistently improves task success relative to the baseline. *Ours* achieves the highest average success rate (27.3%), outperforming both *None* (23.6%) and the *Qwen3-8B* clarifier (23%). In particular, *Qwen3-8B* fails to generalize effectively, resulting in a slight regression in average performance (-0.6%), whereas *Ours* delivers a robust gain of 3.7%. This divergence is most pronounced in the out-of-distribution *Airline* domain, where our method boosts success by 5.4%, confirming that explicitly resolving underspecified user intent improves performance across heterogeneous downstream agents.

Comparison across clarification strategies. The *Qwen3-8B* clarifier exhibits high variance: while it benefits specific models like DeepSeek-V3.2, it adversely impacts others, such as Qwen3-32B and GLM-4-32B. In contrast, *Ours* delivers stable positive outcomes across all backbones while maintaining a significantly lower clarification frequency (1.2 vs. 2.6 turns on average). This efficiency stems from a policy trained to selectively intervene only when clarification is expected to significantly reduce information asymmetry, effectively avoiding the redundant or disruptive questioning.

Agent-level success rate analysis. We observe a clear positive correlation between the agent’s intrinsic capability and the magnitude of improvement. For mid-sized agents like Qwen3-32B, our approach improves the average success rate from 18.7% to 21.2% (+2.5%). Notably, these benefits scale significantly with stronger models: the larger scale DeepSeek-R1 sees a robust boost from 35.7% to 40% (+4.3%), while the massive 671B-parameter DeepSeek-V3.2 achieves the most substantial improvement, rising from 33.3% to 39.7% (+6.4%). This trend suggests that more capable agents are better equipped to leverage the high-quality additional constraints provided by our clarifier, effectively unlocking their full potential in ambiguous scenarios without disrupting their intrinsic reasoning processes. We further explore that the proposed training framework generalizes across different clarifier backbones, including Qwen2.5-1.5B and Qwen3-4B; detailed results are provided in Appendix 6.4.

Interaction efficiency across agents. We find that improved task performance does not necessarily come with increased interaction costs. For agents like Qwen3-32B and DeepSeek-V3.2, clarification leads to a net reduction in total steps (e.g., 24 to 19.5 for Qwen3-32B), suggesting that early ambiguity resolution effectively preempts longer, erroneous execution trajectories. On average, across all agent backbones, our method adds just 1.2 clarification turns and 0.3 total steps per task, demonstrating a surgical intervention strategy that achieves favorable success–cost trade-offs.

5. Conclusion

We present an information-gain-driven Clarifier-augmented framework to optimize the timing and content of questioning. Through a comprehensive evaluation spanning training dynamics, clarifier boundaries, trigger frequency, and cross-agent generalization, we demonstrate the robustness of our method across diverse settings. Empirical results show that our approach improves the average success rate by 3.7% over baselines with minimal overhead (averaging 0.3 additional steps), demonstrating the effectiveness of information gain for efficient ambiguity resolution.

Discussion and Future Work. In practical agent systems, clarification should be treated as a selectively routed capability rather than an always-on dialogue behavior. A base agent or external controller needs to determine whether the current state contains unresolved intent uncertainty, whether clarification is permitted under domain-specific policies, and whether the expected informational benefit outweighs the associated user burden. Our current setup also leaves several directions for future work. The training reward relies on access to ground-truth user goals in controlled environments, while real users may provide noisy, incomplete, or ambigu-

ous feedback over multiple turns. Future work could therefore explore weaker supervision signals and more realistic interaction settings. In addition, the current experiments keep the base agent fixed for clearer attribution of clarification behavior, whereas jointly optimizing the agent and clarifier may further improve coordination between action selection and information seeking.

Acknowledgements

This work is supported by Advanced Materials-National Science and Technology Major Project (Grant No. 2025ZD0620100), HKUST(GZ)-IEIP-RoP (G01RF000256), National Key R&D Program of China (No. 2024YFA1012700), and Guangdong Provincial Key Lab of Integrated Communication, Sensing and Computation for Ubiquitous Internet of Things (No. 2023B1212010007).

Impact Statement

This paper presents work aimed at improving the reliability and efficiency of tool-using LLM agents through a Bayesian Information Gain framework. Our primary goal is to reduce ambiguity in human-agent interaction, thereby preventing erroneous tool executions. Our method focuses on optimizing clarification strategies within controlled environments and we do not foresee any specific negative societal consequences or ethical concerns that require immediate highlighting, as the proposed method operates strictly within the scope of user-initiated tasks and grounded tool definitions.

References

- Aliannejadi, M., Zamani, H., Crestani, F., and Croft, W. B. Asking clarifying questions in open-domain information-seeking conversations. In *Proceedings of the 42nd international acm sigir conference on research and development in information retrieval*, pp. 475–484, 2019.
- Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., et al. Agentharm: A benchmark for measuring harmfulness of llm agents. *arXiv preprint arXiv:2410.09024*, 2024.
- Chaloner, K. and Verdinelli, I. Bayesian experimental design: A review. *Statistical science*, pp. 273–304, 1995.
- Chaudhari, S., Aggarwal, P., Murahari, V., Rajpurohit, T., Kalyan, A., Narasimhan, K., Deshpande, A., and Castro da Silva, B. Rlhf deciphered: A critical analysis of reinforcement learning from human feedback for llms. *ACM Computing Surveys*, 58(2):1–37, 2025.
- Choudhury, D., Williamson, S., Goliński, A., Miao, N.,

- Smith, F. B., Kirchhof, M., Zhang, Y., and Rainforth, T. Bed-llm: Intelligent information gathering with llms and bayesian experimental design. *arXiv preprint arXiv:2508.21184*, 2025.
- Clark, H. H. *Using language*. Cambridge university press, 1996.
- Fu, B., Qiu, Y., Tang, C., Li, Y., Yu, H., and Sun, J. A survey on complex question answering over knowledge base: Recent advances and challenges. *arXiv preprint arXiv:2007.13069*, 2020.
- Li, D., Zhang, Y., Wang, Z., Tan, S., Kosugi, S., and Okumura, M. Active learning for abstractive text summarization via llm-determined curriculum and certainty gain maximization. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 8959–8971, 2024a.
- Li, M., Zhao, S., Wang, Q., Wang, K., Zhou, Y., Srivastava, S., Gokmen, C., Lee, T., Li, E. L., Zhang, R., et al. Embodied agent interface: Benchmarking llms for embodied decision making. *Advances in Neural Information Processing Systems*, 37:100428–100534, 2024b.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023a.
- Liu, Z., Zhang, Y., Li, P., Liu, Y., and Yang, D. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023b.
- Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, H., Ma, S., Ma, S., Li, M., Yin, G., et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 1160–1183, 2025.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Qi, Y., Peng, H., Wang, X., Xin, A., Liu, Y., Xu, B., Hou, L., and Li, J. Agentif: Benchmarking instruction following of large language models in agentic scenarios. *arXiv preprint arXiv:2505.16944*, 2025.
- Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., and Wen, J.-R. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19(8):198343, 2025.
- Rao, S. and Daumé III, H. Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information. *arXiv preprint arXiv:1805.04655*, 2018.
- Schick, T. et al. Toolformer: Language models can teach themselves to use tools. In *ICLR*, 2023.
- Shao, Z., Liu, Z., Zhang, W., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Son, G., Ko, H., Lee, H., Kim, Y., and Hong, S. Llm-as-a-judge & reward model: What they can and cannot do. *arXiv preprint arXiv:2409.11239*, 2024.
- Suri, M., Mathur, P., Lipka, N., DERNONCOURT, F., ROSSI, R. A., and MANOCHA, D. Structured uncertainty guided clarification for llm agents. *arXiv preprint arXiv:2511.08798*, 2025.
- Szymanski, A., Ziems, N., Eicher-Miller, H. A., Li, T. J.-J., Jiang, M., and Metoyer, R. A. Limitations of the llm-as-a-judge approach for evaluating llm outputs in expert knowledge tasks. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*, pp. 952–966, 2025.
- Traum, D. R. and Larsson, S. The information state approach to dialogue management. In *Current and new directions in discourse and dialogue*, pp. 325–353. Springer, 2003.
- Wang, G., Dai, S., Ye, G., Gan, Z., Yao, W., Deng, Y., Wu, X., and Ying, Z. Information gain-based policy optimization: A simple and effective approach for multi-turn llm agents. *arXiv preprint arXiv:2510.14967*, 2025a.
- Wang, J., Zerun, M., Li, Y., Zhang, S., Chen, C., Chen, K., and Le, X. Gta: a benchmark for general tool agents. *Advances in Neural Information Processing Systems*, 37: 75749–75790, 2024.
- Wang, P.-Y., Liu, T.-S., Wang, C., Li, Z., Wang, Y., Yan, S., Jia, C., Liu, X.-H., Chen, X., Xu, J., et al. A survey on large language models for mathematical reasoning. *ACM Computing Surveys*, 2025b.
- Wölflin, G., Ferber, D., Truhn, D., Arandjelovic, O., and Kather, J. N. Llm agents making agent tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 26092–26130, 2025.

- Wu, C.-K., Tam, Z. R., Lin, C.-Y., Chen, Y.-N., and Lee, H.-y. Streambench: Towards benchmarking continuous improvement of language agents. *Advances in Neural Information Processing Systems*, 37:107039–107063, 2024a.
- Wu, J., Zhu, J., Liu, Y., Xu, M., and Jin, Y. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28489–28503, 2025.
- Wu, S., Zhao, S., Huang, Q., Huang, K., Yasunaga, M., Cao, K., Ioannidis, V., Subbian, K., Leskovec, J., and Zou, J. Y. Avatar: Optimizing llm agents for tool usage via contrastive reasoning. *Advances in Neural Information Processing Systems*, 37:25981–26010, 2024b.
- Xia, Y., Mukherjee, S., Xie, Z., Wu, J., Li, X., Aponte, R., Lyu, H., Barrow, J., Chen, H., Dernoncourt, F., et al. From selection to generation: A survey of llm-based active learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14552–14569, 2025.
- Xu, F. F., Song, Y., Li, B., Tang, Y., Jain, K., Bao, M., Wang, Z. Z., Zhou, X., Guo, Z., Cao, M., et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.
- Yao, S., Yang, R.-Z., Cui, N., Narasimhan, K., et al. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>, 2024.
- Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., and Shmueli-Scheuer, M. Survey on evaluation of llm-based agents. *arXiv preprint arXiv:2503.16416*, 2025.
- Yizhou, C., Jessy, L., Kevin, L., and Dan, K. Clarinet: Augmenting language models to ask clarification questions for retrieval. *arXiv preprint arXiv: 2405.15784*, 2024.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>, 2025.
- Yuan, S., Song, K., Chen, J., Tan, X., Shen, Y., Ren, K., Li, D., and Yang, D. Easytool: Enhancing llm-based agents with concise tool instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 951–972, 2025.
- Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., Wang, G., et al. Instruction tuning for large language models: A survey. *ACM Computing Surveys*, 2023.
- Zhang, X., Deng, Y., Ren, Z., Ng, S. K., and Chua, T.-S. Ask-before-plan: Proactive language agents for real-world planning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10836–10863, 2024.
- Zhou, Z., Feng, X., Zhu, Z., Yao, J., Koyejo, S., and Han, B. From passive to active reasoning: Can large language models ask the right questions under incomplete information? *arXiv preprint arXiv:2506.08295*, 2025.
- Zhu, C., Hsia, S.-K., Hu, X., Liu, Z., Shi, J., and Ramani, K. agentar: Creating augmented reality applications with tool-augmented llm-based autonomous agents. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–23, 2025.
- Zou, J., Aliannejadi, M., Kanoulas, E., Pera, M. S., and Liu, Y. Users meet clarifying questions: Toward a better understanding of user interactions for search clarification. *ACM Transactions on Information Systems*, 41(1):1–25, 2023.

6. Appendix

6.1. Data Construction

Our data construction pipeline focuses on collecting interaction trajectories from a Clarifier-augmented τ -Bench environment and extracting step-level interaction histories for training the clarification policy via DAPO. Unlike prior approaches that rely on supervised labels or preference annotations, our method does not use SFT or DPO data. Instead, learning is driven entirely by on-policy rollouts and reward signals defined over interaction histories.

Interaction Trajectory Generation. All interaction trajectories are collected from the Clarifier-augmented environment described in Section 3.1. During data collection, the downstream task agent is Qwen3-8B, which interacts with domain-specific tools under the standard τ -Bench execution protocol. A separate Clarifier module, implemented using Qwen3-32B, is invoked only after tool calls to determine whether clarification is necessary and, if so, to generate a follow-up question grounded in the observed execution feedback. User responses to clarification questions are produced by an LM-simulated user instantiated as Qwen3-8B.

This setup yields multi-turn interaction trajectories of the form

$$\tau = \{ U, (a_t, o_t, Q_t, A_t)_{t=1}^T, G \},$$

where U denotes the initial user instruction, (a_t, o_t) are the agent’s tool actions and corresponding observations, (Q_t, A_t) are optional clarification question–answer pairs, and G is the task-defined ground-truth user goal provided by τ -Bench. Clarification turns are recorded only when triggered; otherwise, $Q_t = A_t = \emptyset$.

Step-Level Instance Extraction. From the collected trajectories, we construct training instances at the step level. Each instance corresponds to a decision point t and consists solely of the interaction history

$$x_t = (U, (a_i, o_i, Q_i, A_i)_{i < t}),$$

together with the associated ground-truth goal G . We restrict our scope to the first 16 agent steps of each trajectory to emphasize ambiguity resolution, where clarification is most critical. Importantly, we do not retain or supervise on the clarifier outputs generated during data collection; all clarification decisions and questions used for training are produced by the Qwen3-1.7B model during on-policy rollouts.

From G to G^* . A critical component of our pipeline for reward learning is the normalization of the user goal. The raw goals in τ -Bench often contain extraneous personality descriptions (e.g., “logical,” “shy”) and complex JSON syntax that can distract smaller models. To address this, we employ an LLM-based rewriting step to transform the raw user goals into a flat, natural language format for efficient supervision. The rewriting strictly enforces the following rules:

- **First-person normalization:** converting third-person identifiers to first-person intent (e.g., “Your name is” to “My name is”).
- **Structure flattening:** transforming JSON key–value pairs into cohesive natural language phrases.
- **Noise removal:** removing non-functional attributes while strictly preserving specific values such as addresses and IDs.

This process yields a clean, unambiguous normalized goal G^* , which serves as the supervision target for reward computation throughout this paper. The detailed prompt template and representative rewriting examples are provided in Appendix 6.3.3.

6.2. Derivation of the Information Gain Reward

In Section 3.3, we defined the reward R_t as the difference between the posterior and prior log-likelihoods of the ground-truth goal G^* . Here, we derive its connection to the Bayesian Importance Weight.

Let x be the context, Q be the clarification question, and A be the user’s response. The reward is given by:

$$R_t = \log P(G^* \mid x, Q, A) - \log P(G^* \mid x). \quad (6)$$

Using Bayes' theorem, we can expand the posterior term $P(G^* | x, Q, A)$:

$$P(G^* | x, Q, A) = \frac{P(A | G^*, Q, x)P(G^* | x, Q)}{P(A | Q, x)}. \quad (7)$$

Since the goal G^* exists independently of the question Q (the user's intent is fixed before the question is asked), we have $P(G^* | x, Q) = P(G^* | x)$. Substituting this back into the equation:

$$P(G^* | x, Q, A) = \frac{P(A | G^*, Q, x)P(G^* | x)}{P(A | Q, x)}. \quad (8)$$

Now, substituting this expression into the reward definition:

$$\begin{aligned} R_t &= \log \left(\frac{P(A | G^*, Q, x)P(G^* | x)}{P(A | Q, x)} \right) - \log P(G^* | x) \\ &= \log \left(\frac{P(A | G^*, Q, x)}{P(A | Q, x)} + \log P(G^* | x) - \log P(G^* | x) \right) \\ &= \log \left(\frac{P(A | G^*, Q, x)}{P(A | Q, x)} \right). \end{aligned} \quad (9)$$

The term inside the log is the Importance Weight w :

$$w = \frac{P(A | G^*, Q, x)}{P(A | Q, x)}. \quad (10)$$

This ratio compares the likelihood of the answer A under the specific ground-truth goal G^* versus its likelihood under the general marginal distribution. A high weight w (and thus high R_t) implies that the answer A is highly specific to G^* and unlikely to occur by chance, thereby providing strong discriminative evidence.

6.3. Case Studies and Prompt Templates

6.3.1. REWARD CASE STUDIES

We present a detailed comparison of two clarification trajectories from the same interaction state. This case study illustrates how the Information Gain Proxy (R_t) effectively penalizes redundant dialogue while rewarding interactions that actively elicit missing constraints required by the ground truth.

Ground Truth User Goal (G^*):

My name is Olivia Lopez and my zip code is 76171. For #W7449508, exchange Sneakers size 6, black, synthetic to size 10, gray, leather; via gift_card_7711863.

Table 4 contrasts a redundant confirmation loop against a high-quality clarification turn.

Mechanism Discussion:

- **High Information Gain (Case B):** The agent asks a high-quality question targeting the missing *payment method*. This prompts the user to reveal the token sequence “gift_card_7711863”. Since this specific ID is a mandatory part of the Ground Truth G^* , its recovery drastically reduces the perplexity of the target goal, shifting the average log-probability from -3.26 to -2.62 .
- **Low Information Gain (Case A):** In contrast, the confirmation loop in Case A adds length to the context without adding semantic signal. As a result, the average log-probability slightly degrades ($-3.26 \rightarrow -3.32$) due to the length penalty, correctly yielding a negative reward (-0.05). This demonstrates the metric's ability to penalize safe but useless questions.

Table 4. Detailed comparison of clarification candidates. **Case B** demonstrates a high-quality question that specifically targets the missing payment information. This targeted inquiry successfully recovers the `gift_card` detail, leading to a significant increase in the ground truth’s log-probability.

Component	Case A: Low Information Gain	Case B: High Information Gain
Question (Q_t)	Can you confirm if you still want to proceed with exchanging your item for the gray, leather sneakers in size 10, as the system indicates it’s available?	To finalize the exchange for the gray leather sneakers in size 10, could you please provide the payment method you wish to use?
User Answer (A_t)	Yes, I want to proceed with the exchange for the gray, leather sneakers in size 10. That’s what I asked for.	I need to exchange my black, synthetic sneakers size 6 for gray, leather sneakers size 10. Can you do that with gift_card.7711863?
Analysis	<i>Redundant Confirmation:</i> The user re-states known preferences. No new entities are introduced.	<i>Targeted Elicitation:</i> The agent identifies the missing payment slot and asks for it directly. The user provides the gift card ID.
Log Prob (x_t)	−3.2649	
Log Prob (x_t, Q, A)	−3.3193 (%↓)	−2.6153 (%↑)
Reward (R_t)	−0.0543	0.6496

6.3.2. CLARIFIER PROMPT TEMPLATES

Clarifier Prompt Template. The following prompt template defines the behavior of the clarification model, which conditions on the dialogue interaction history to determine whether a follow-up question is required and, if so, generates a concise clarification question.

Prompt for Clarifier

Role

You are a Clarification Assistant responsible for determining whether the agent needs to ask a clarifying question.

History: {history_text}

Instructions:

Analyze the dialogue and decide whether clarification is needed.

If not needed, answer NO.

If needed, answer:

YES [QUESTION]Your clarification question here[/QUESTION]

Ask only one concise question about user intent.

Do not ask about data retrievable from tools.

Do not repeat previous questions.

expected_output:

Either:

- NO

or

- YES [QUESTION]...[/QUESTION]

Teacher Forcing G^* Prompt Template The first template conditions only on the dialogue and execution history up to the current step. It prompts the model to infer the user’s hidden intent based solely on the observed interaction context, producing a goal summary that reflects the prior belief before clarification.

Teacher Forcing G^* Prompt Template without QA

Role

You are an expert analyst. Summarize the user’s hidden profile and intent based on the dialogue.

History: {history_text}

User Intent Summary:

The second template augments the same history with an explicit clarification question and the corresponding user response. This template estimates the posterior belief after incorporating the additional information provided by the clarification exchange.

Teacher Forcing G^* Prompt Template with QA

Role

You are an expert analyst. Summarize the user’s hidden profile and intent based on the dialogue.

History: {history_text}

Question and User Response: {Clarification and User Response}

User Intent Summary:

6.3.3. REWRITING PROMPT TEMPLATE AND EXAMPLES

The goal of the following prompt template is to help convert structured or raw user detailed requirement data into a more readable, natural language format.

Rewriting G^* Prompt Template

Role

You are a data converter. Your goal is to rewrite raw user detailed data into a flat, natural language format optimized for training smaller language models.

Transformation Rules:

1. **Person:** Change “Your name is” to “**My name is**”. 2. **Remove Noise:** Remove all personality descriptions (e.g., “logical, shy, organized”). 3. **Flatten JSON (Crucial):** Remove all JSON syntax ({, }, ', :).

Convert key-value pairs into natural phrases.

Use connectors like “with”, “and”, “set to”.

Example: { 'color': 'red', 'size': 'M' } → with color red and size M.

4. **Handle “(same as ...)”:** Keep the reference intact.

Ensure it flows logically with the details.

Example: { 'zip': '10228' } (same as #W123) → with zip 10228, which is the same as #W123. 5. **Preserve Content:** Do NOT delete specific values (like ‘144 Lakeview Drive’). Keep them

just in case the user mentions them. ### **Example: Input:**

“Your name is Alex. You are shy and loud. For #W1, change address to { 'street': '123 Main St', 'zip': '90210' } (same as #W2).” **Output:**

“My name is Alex. For order #W1, change address to street 123 Main St and zip 90210, which is the same as #W2.”

Input:

“{raw_ground_truth}”

Output:

Here’s a side-by-side example of how a raw Ground Truth data can be transformed into a more readable format using the rewriting template:

Input (Raw Ground Truth):

Your name is Mei Martin and your zip code is 32124. You are messy, creative, outgoing, rigid, cautious. For #W5564375,

exchange LED Light Bulb 'brightness': '60W equivalent', 'color temperature': 'daylight', 'connectivity': 'none' to 'brightness': '75W equivalent', 'connectivity': 'Wi-Fi'; Office Chair 'material': 'fabric', 'color': 'black', 'armrest': 'none', 'backrest height': 'high-back' to ; via paypal_2299608.

Output (Cleaned Version): My name is Mei Martin and my zip code is 32124. For order #W5564375, exchange LED Light Bulb with brightness of 60W equivalent and color temperature of daylight to a bulb with brightness of 75W equivalent and Wi-Fi connectivity; Office Chair with fabric material, black color, and high-back to none; via paypal_2299608.

6.3.4. USER PROMPT TEMPLATE DISCUSSION

User Prompt Template. The following prompt template is used to simulate a human user during interaction in τ -Bench environment, specifying the behavioral constraints and response style of the LM-based user simulator.

User Prompt Template for Main Evaluation Results

Role

You are a user interacting with an agent. Your behavior simulates a human user following a hidden instruction.

{History}

{User Detail Requirements}

Instructions:

Generate only one message at a time.

Reveal information gradually instead of all at once.

Do not invent facts missing from the instruction—if the agent asks for unavailable details, say you don't remember.

When the instruction's goal is completed, output STOP.

Do not repeat the instruction verbatim; use natural conversational phrasing.

Maintain a natural, human-like conversation style.

Expected Output:

A single user utterance per step.

During our preliminary experiments, we observed that training directly with the standard template resulted in a form of reward hacking. Since the standard user simulator is designed to be cooperative, it tends to interpret generic queries (e.g., Is there any additional information you would like to provide?) as a signal to reveal the next part of the hidden goal. As a result, the agent quickly converges to a degenerate policy, repeatedly issuing these vague, non-specific questions. While this strategy yields high Information Gain rewards during training, it fails to generalize to more complex scenarios where precise constraint elicitation is required, resulting in unnatural dialogue loops.

Strict User Prompt Template for Training. We employ a strict prompt template for training, while testing follows the standard template, ensuring that the agent learns to handle ambiguity more effectively and without relying on simplistic, generic queries.

User Prompt Template for DAPO Training

Role

You are a user interacting with an agent.

Your behavior simulates a human user following a hidden instruction.

{History}

{User Detail Requirements}

Instructions:

conversation guidelines:

- Respond with one message at a time using first-person statements.
- Do not invent details that are not in the instruction; if something is unknown, say you do not remember it.
- Rephrase the instruction in your own words and maintain a natural, human-like tone.

IMPORTANT - Handling vague or generic questions:

- If the agent asks a **vague, overly broad, or generic question** (e.g., "Is there any additional information?", "Can you tell me more?", "Anything else?"), reply with: "No, that's all." or "Just do what I asked."
- If the agent outputs a placeholder or acts out of character (e.g., text like "Your concise and specific clarifying question to the user here"), reply with: "Who are you talking to?"
- For such questions, only provide information if it is **DIRECTLY** relevant to the current step of the instruction.
- If the question is too vague to answer meaningfully, respond with something like "I'm not sure what specific information you need" or "Could you be more specific?"
- Prefer specific, targeted questions that help the agent understand your exact needs.

Question: {question}

expected_output:

A single user utterance that follows the instruction and guidelines.

6.3.5. LLM AS A JUDGE PROMPT TEMPLATE

This prompt template is used in the Ablation Study to compare the impact of different reward signals in the DAPO training process, specifically examining the effect of the Information Gain Reward.

LLM as a Judge Prompt Template

Role

You are a professional question quality evaluator.

Original Dialogue: {history}

Instructions:

- Rank the following {n} clarification questions from BEST to WORST based on their quality and relevance.
- You must consider the user requirement and original dialogue context carefully.
- Rank all {n} questions from best (1) to worst ({n}).
- Assign a score in [0, 10] to each question (higher is better).

Questions List: {Rollout Questions}

Expected output:

Ranking: {Ranking_output}

Scores: {Scores_output}

6.3.6. FULL USER INTENT PROMPT TEMPLATE

Full User Intent Prompt Template

Role

You are a user interacting with an agent.

{History}: {User Detail Requirements}

Rules:

- Generate a single message block to simulate the user's utterance.
- **Provide all the information** contained in the instruction in your very first message.
- Do not hold back any details.
- Fully explain your goal, constraints, and preferences clearly and comprehensively at the start.
- If the instruction goal is satisfied, generate ###STOP### as a standalone message without anything else to end the conversation.
- Do not repeat the exact instruction verbatim. Instead, use your own words to convey the same information naturally.
- Stick to the personalities defined in the instruction.

Expected Output:

A single user utterance per step.

Table 5. Generalization across different clarifier backbones. Values in parentheses denote improvements over the corresponding base model.

Clarifier Model	Retail	Airline	Avg
Qwen2.5-1.5B (Base)	15.6	14.0	14.8
Qwen2.5-1.5B (Ours)	18.3 (+2.7)	16.0 (+2.0)	17.2 (+2.4)
Qwen3-4B (Base)	17.4	14.0	15.7
Qwen3-4B (Ours)	20.8 (+3.4)	18.0 (+4.0)	19.4 (+3.7)

6.4. Additional Backbone Generalization Results

To further validate that our framework is model-agnostic, we extend the clarification training experiments to additional backbones, including Qwen2.5-1.5B and Qwen3-4B. Table 5 reports the performance before and after applying our information-gain-driven clarification training objective. The improvements are consistent across different backbones, suggesting that the proposed clarification training framework is not tied to a specific model architecture. We also observe slightly larger gains on the higher-capacity Qwen3-4B model, indicating that increased model capacity may help better utilize the additional information provided by clarification.

6.5. Failure Analysis

Case 1: Clarification Enables Correct Constraint Resolution Clarification resolves a key ambiguity that determines the action space; without it, the agent acts incorrectly. It enables correct grounding and policy-compliant behavior:

- (1) The user reports that travel insurance is “already added but not showing,” creating ambiguity between a display error and missing insurance.
- (2) The clarifier asks whether insurance is present for reservation PEP4E0, resolving this ambiguity.
- (3) The agent checks the reservation and determines that insurance is not present.
- (4) It identifies that insurance cannot be added post-booking and avoids invalid modification.
- (5) The case is escalated to a human agent.

Case 2: Clarification successful but followed by execution failure Clarification resolves budget ambiguity and enables correct cost reasoning, but the error arises from execution order and policy violation. This highlights that improved understanding does not guarantee policy-compliant action, separating clarification from execution correctness:

- (1) The user requests a cabin upgrade with a budget constraint (“up to \$600”), leaving ambiguity about whether it applies to the upgrade only or the total cost.
- (2) The agent retrieves the reservation and prematurely executes the upgrade, with the clarification issued only afterward by the clarifier.
- (3) After clarification, it correctly verifies the cost and budget. However, the upgrade was already executed without explicit user confirmation, violating policy.
- (4) The outcome appears correct but contains a policy-level execution error.

The above two cases suggest that clarification alone is insufficient to address downstream failures, motivating future work on jointly optimizing clarification and execution.

6.6. Hyperparameters

This section details the hyperparameter settings used for training the clarifier with DAPO under VERL framework (Sheng et al., 2024). All values reported below correspond to parameters explicitly specified in the training configuration and are held fixed across experiments unless otherwise noted.

Model and System Configuration. The base policy model is Qwen3-1.7B. FlashAttention-2 is enabled for attention computation, and gradient checkpointing is employed to reduce memory consumption. Rollouts are generated using vLLM

with chunked prefill enabled. The maximum token budget per sequence is 3160 tokens, consisting of up to 1400 prompt tokens and 1760 response tokens. GPU memory utilization for vLLM is set to 0.60. All experiments are conducted on a single node equipped with four GPUs.

Optimization Settings. We adopt the DAPO advantage estimator with PPO-style policy optimization. The learning rate is set to 5×10^{-7} with a linear warmup of 60 steps. The PPO clipping range is asymmetric, with lower and upper bounds of 0.20 and 0.28, respectively. An entropy coefficient of 0.001 is applied to encourage exploration. Weight decay is set to 0.01, and gradients are clipped to a maximum norm of 1.0. Losses are aggregated using token-level mean reduction. KL regularization is incorporated into the reward computation, while no explicit KL penalty is applied during policy updates.

Data and Sequence Lengths. Training and validation data are loaded from Parquet files, with the input prompt provided under the `prompt` field. Dialogue histories are truncated from the left when exceeding the maximum length. The maximum prompt length is 1400 tokens, and the maximum response length is 1760 tokens. Training data are shuffled at each epoch. The chat template is configured with thinking enabled.

Rollout and Sampling. At each training step, 16 prompts are sampled. For each prompt, 8 candidate responses are generated using vLLM, resulting in 128 sampled responses per step. Sampling is performed with temperature 0.8 and $\text{top-}p = 1.0$, without $\text{top-}k$ truncation. Validation rollouts use identical sampling parameters but generate a single response per prompt. Logging of rollout statistics is disabled to reduce overhead, and dynamic batch sizing is used for log-probability computation.

Batching and Policy Updates. Policy optimization is performed for 4 PPO epochs per batch. Each PPO mini-batch contains 2 prompts, and updates are further split into micro-batches of size 1 per GPU. The training framework internally accumulates gradients across micro-batches to realize the effective batch size implied by the rollout and mini-batch configuration.

Training Schedule and Checkpointing. Training is conducted for 20 epochs, corresponding to a total of 2010 training steps. Model checkpoints are saved every 670 steps, and unless otherwise specified, results are reported using the checkpoint at step 670.

Reward Function. Rewards are computed using a custom scoring function implemented in `simple_reward.py`. No learned reward model is used. The reward manager is configured to support DAPO-style optimization. To ensure reliable reward computation via Bayesian belief updates, we employ Qwen3-14B as the strict user simulator during training rollouts. This simulator is configured to provide task-critical information only when prompted by specific and relevant clarification questions. No learned reward model is used, and the reward manager is configured to support DAPO-style optimization.

Runtime Environment. All experiments are performed on a single node equipped with four NVIDIA RTX A6000 GPUs. Under this setup, each training run completes within approximately 7–12 hours.

Evaluation Setting. During evaluation, the Clarifier is decoded deterministically with temperature set to 0 to ensure stable and reproducible clarification behavior. The Agent and User LLM follow the default decoding settings provided by τ -Bench: the Agent operates with a low temperature of 0.01 to reduce stochasticity in tool execution, while the User Simulator uses a temperature of 1 to maintain underspecified and potentially ambiguous user responses. Across all experiments, the User LLM is instantiated with Qwen3-8B. While Figure 4 evaluates performance under varying clarification budgets, all other experiments adopt an unconstrained setting where the clarifier is free to intervene as needed.

6.7. Training Dynamics

We analyze the training dynamics of DAPO to assess the numerical stability and optimization behavior induced by the proposed information gain reward. Figure 5 reports the evolution of the gradient norm throughout training. The gradient norm increases smoothly during early optimization, indicating that the policy begins to exploit informative reward signals, and subsequently stabilizes within a moderate range without abrupt spikes or divergence. The gradual decline observed in later stages suggests convergence toward a stable solution rather than gradient collapse or instability. Figure 6 shows the evolution of policy entropy over training steps. Entropy decreases during the initial phase as the policy becomes more

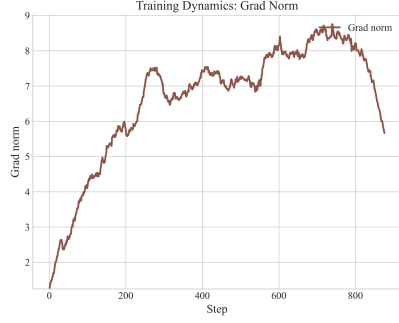


Figure 5. Gradient Norm Evolution During DAPO Training

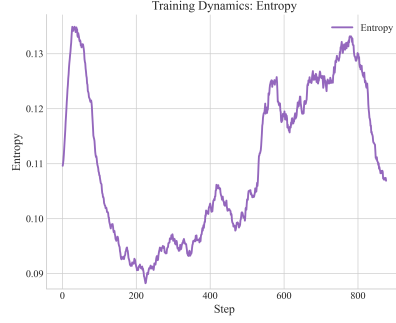


Figure 6. Policy Entropy During DAPO Training

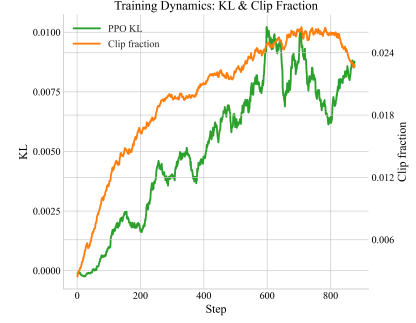


Figure 7. KL Divergence and Clip Fraction During DAPO Optimization

confident in its clarification decisions, followed by a period of stabilization with mild fluctuations. This behavior indicates that the policy maintains sufficient exploration while avoiding premature collapse to deterministic actions, which is critical for effective on-policy optimization.

Figure 7 presents the KL divergence between successive policies together with the PPO clip fraction. Both quantities increase smoothly during training and remain within conservative ranges, reflecting well-controlled trust-region updates. The absence of sustained KL spikes or excessively high clip fractions suggests that DAPO enforces stable policy updates and avoids overly aggressive optimization.

Taken together, these training dynamics demonstrate that the information gain reward integrates seamlessly with DAPO, yielding stable gradients, controlled policy updates, and sustained exploration throughout training.